

Pack 17 (Desktop) — Safe Host Input Executor Shell + Per-Action Rate Limiting + Session Audit Hooks (real code)

RemoteDesk · Pack 17 (Desktop) — Safe Host Input Executor Shell + Per-Action Rate Limiting + Session Audit Hooks (real code)

Codex / Claude Code handoff. Direct continuation of Desktop Packs 15 + 16. This slice adds the missing piece on the HOST side: a safe, abstracted input-executor shell that every gate funnels through (still no-op for native OS by default), per-action rate limiting to stop input floods, and structured session audit hooks for the critical permission/control events. Build order: shared event contract → executor shell → rate limiter → audit hooks → wiring → UI counters → tests.

Scope: give the host a single, auditable choke point for inbound input that respects every Pack 15/16 gate, throttles abusive input, and emits non-sensitive audit events. **Native OS execution stays disabled by default behind an explicit, off-by-default capability flag. Fail-closed everywhere. No secrets in logs.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]

```
packages/shared/src/permissions/inputAction.ts
packages/shared/src/permissions/sessionAuditEvents.ts
apps/desktop/src/renderer/src/services/hostInputExecutor.ts
apps/desktop/src/renderer/src/services/inputRateLimiter.ts
apps/desktop/src/renderer/src/services/sessionAuditHooks.ts
apps/desktop/src/renderer/src/components/InputActivityMeter.tsx
tests/permissions/inputRateLimiter.test.ts
tests/permissions/hostInputExecutor.test.ts
tests/permissions/sessionAuditEvents.test.ts
```

MODIFIED [MODIFY]

```
packages/shared/src/permissions/index.ts      (export inputAction + audit events)
apps/desktop/src/renderer/src/services/webRTC.ts (route inbound input through executor)
apps/desktop/src/renderer/src/services/emergencyStop.ts (emit audit on engage/release)
apps/desktop/src/renderer/src/components/RemoteSessionView.tsx (input activity meter)
```

NOT TOUCHED (intentionally)

```
apps/desktop/src/main/index.ts      (no real native executor enabled)
apps/desktop/src/preload/index.ts    (no new privileged bridge)
apps/desktop/src/renderer/src/services/socket.ts
apps/desktop/src/renderer/src/services/permissionState.ts (Pack 15/16 unchanged)
apps/api/** apps/web/**
```

Part A — Shared: input action + audit event contracts

FILE: packages/shared/src/permissions/inputAction.ts

```
/**
 * Normalized inbound input action. The HOST executor reasons about these, not raw
 * DOM/native events, so gating + rate limiting + auditing are payload-agnostic and
 * never need to inspect (or log) actual key codes or coordinates.
 */
export type InputActionKind =
  | 'mouse_move'
  | 'mouse_button'
  | 'mouse_scroll'
  | 'key'
  | 'text';

export interface InputAction {
  kind: InputActionKind;
  /** Opaque, already-validated payload. Treated as a black box by gates/audit. */
  payload: unknown;
  /** Monotonic client sequence for ordering/dedupe (no timing secrets). */
  seq: number;
}

/** Which kinds are "high-frequency" and should be rate-limited more loosely. */
export const HIGH_FREQUENCY_KINDS: readonly InputActionKind[] = ['mouse_move', 'mouse_scroll'];

export function isHighFrequency(kind: InputActionKind): boolean {
  return HIGH_FREQUENCY_KINDS.includes(kind);
}

/** A non-sensitive descriptor for logs/audit: kind + seq only, never the payload. */
export function describeAction(action: InputAction): { kind: InputActionKind; seq: number } {
  return { kind: action.kind, seq: action.seq };
}
```

FILE: packages/shared/src/permissions/sessionAuditEvents.ts

```
/**
 * Session-local audit events for control/permission changes. These are
 * NON-SENSITIVE by construction: no payloads, no keystrokes, no tokens. They can
 * be forwarded to the API audit log (Pack 14) by the host if desired.
 */
export type SessionAuditType =
  | 'control.granted'
  | 'control.revoked'
  | 'emergency_stop.engaged'
  | 'emergency_stop.released'
  | 'input.blocked'
  | 'input.rate_limited'
  | 'session.expired';

export interface SessionAuditEvent {
  type: SessionAuditType;
  /** ISO timestamp. */
  at: string;
  /** Small, non-sensitive detail (counts, reasons, kinds). NEVER payloads. */
}
```

```

    detail: Record<string, string | number | boolean>;
  }

export function makeAuditEvent(
  type: SessionAuditType,
  detail: Record<string, string | number | boolean> = {},
  now: Date = new Date(),
): SessionAuditEvent {
  return { type, at: now.toISOString(), detail };
}

/** Guard: reject any detail key that looks sensitive (defense in depth). */
const FORBIDDEN = [/token/i, /password/i, /secret/i, /authorization/i, /cookie/i, /key$/i, /payload/i];

export function isAuditDetailSafe(detail: Record<string, unknown>): boolean {
  return !Object.keys(detail).some((k) => FORBIDDEN.some((re) => re.test(k)));
}

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

export * from './devicePolicy';
export * from './policyEvaluator';
export * from './sessionApproval';
export * from './inputAction';          // ADD
export * from './sessionAuditEvents';  // ADD

```

Part B — Desktop: input rate limiter

FILE: apps/desktop/src/renderer/src/services/inputRateLimiter.ts

```

import { isHighFrequency, type InputActionKind } from '@remotedesk/shared';

/**
 * Token-bucket rate limiter for inbound input actions. High-frequency kinds
 * (mouse move/scroll) get a larger bucket; discrete kinds (key/text/button) get a
 * tighter one to blunt scripted floods. Pure + deterministic given a clock.
 */
export interface RateLimiterOptions {
  /** Tokens per second for discrete actions (key/button/text). */
  discreteRatePerSec?: number;
  /** Tokens per second for high-frequency actions (move/scroll). */
  highFreqRatePerSec?: number;
  /** Max burst (bucket capacity) multiplier applied to the per-sec rate. */
  burstFactor?: number;
}

interface Bucket {
  tokens: number;
  capacity: number;
  ratePerMs: number;
  lastRefill: number;
}

export class InputRateLimiter {
  private discrete: Bucket;

```

```

private highFreq: Bucket;

constructor(opts: RateLimiterOptions = {}, now: number = Date.now()) {
  const discreteRate = opts.discreteRatePerSec ?? 25;
  const highFreqRate = opts.highFreqRatePerSec ?? 200;
  const burst = opts.burstFactor ?? 2;
  this.discrete = this.makeBucket(discreteRate, burst, now);
  this.highFreq = this.makeBucket(highFreqRate, burst, now);
}

/** Returns true if the action is allowed; false if it should be dropped. */
allow(kind: InputActionKind, now: number = Date.now()): boolean {
  const bucket = isHighFrequency(kind) ? this.highFreq : this.discrete;
  this.refill(bucket, now);
  if (bucket.tokens >= 1) {
    bucket.tokens -= 1;
    return true;
  }
  return false;
}

private makeBucket(ratePerSec: number, burstFactor: number, now: number): Bucket {
  const capacity = ratePerSec * burstFactor;
  return { tokens: capacity, capacity, ratePerMs: ratePerSec / 1000, lastRefill: now };
}

private refill(bucket: Bucket, now: number): void {
  const elapsed = now - bucket.lastRefill;
  if (elapsed <= 0) return;
  bucket.tokens = Math.min(bucket.capacity, bucket.tokens + elapsed * bucket.ratePerMs);
  bucket.lastRefill = now;
}
}

```

Part C — Desktop: safe host input executor shell

FILE: apps/desktop/src/renderer/src/services/hostInputExecutor.ts

```

import {
  describeAction,
  makeAuditEvent,
  type InputAction,
  type SessionAuditEvent,
} from '@remotedesk/shared';
import type { RemoteInputGate } from '../remoteInputGate';
import { InputRateLimiter } from '../inputRateLimiter';

/**
 * Optional native sink. By DEFAULT there is none, so execution is a safe no-op.
 * A real OS executor may be injected ONLY if the project already ships one; this
 * shell never imports or enables native input itself.
 */
export interface NativeInputSink {
  execute(action: InputAction): void;
}

```

```

export interface HostInputExecutorOptions {
  /** Inject a real native sink ONLY if a safe one exists. Default: none (no-op). */
  nativeSink?: NativeInputSink;
  rateLimiter?: InputRateLimiter;
  /** Emit audit events (engage/block/rate-limit). */
  onAudit?: (event: SessionAuditEvent) => void;
}

/**
 * The single host-side choke point for inbound input. Every inbound action MUST
 * go through execute(). Order of checks (all fail-closed):
 * 1. policy/permission gate (Pack 15 RemoteInputGate)
 * 2. rate limit
 * 3. native sink (no-op unless explicitly injected)
 * Blocked + rate-limited actions are counted and audited (kind + seq only).
 */
export class HostInputExecutor {
  private readonly rateLimiter: InputRateLimiter;
  private executed = 0;
  private blocked = 0;
  private rateLimited = 0;

  constructor(
    private readonly gate: RemoteInputGate,
    private readonly opts: HostInputExecutorOptions = {},
  ) {
    this.rateLimiter = opts.rateLimiter ?? new InputRateLimiter();
  }

  /** Process one inbound action. Returns the disposition for callers/tests. */
  execute(action: InputAction, now: number = Date.now()): 'executed' | 'blocked' | 'rate_limited' {
    // 1. Policy/permission gate — fail closed.
    if (!this.gate.guardInbound()) {
      this.blocked += 1;
      this.emitBlocked('policy', action);
      return 'blocked';
    }
    // 2. Rate limit.
    if (!this.rateLimiter.allow(action.kind, now)) {
      this.rateLimited += 1;
      this.emitRateLimited(action);
      return 'rate_limited';
    }
    // 3. Native sink — NO-OP unless a safe executor was explicitly injected.
    this.opts.nativeSink?.execute(action);
    this.executed += 1;
    return 'executed';
  }

  stats(): { executed: number; blocked: number; rateLimited: number } {
    return { executed: this.executed, blocked: this.blocked, rateLimited: this.rateLimited };
  }

  private emitBlocked(reason: string, action: InputAction): void {
    if (this.blocked === 1 || this.blocked % 50 === 0) {
      this.opts.onAudit?.(makeAuditEvent('input.blocked', { reason, ...describeAction(action), blockedTotal: this.blocked

```

```

    }));
  }
}

private emitRateLimited(action: InputAction): void {
  if (this.rateLimited === 1 || this.rateLimited % 100 === 0) {
    this.opts.onAudit?.(makeAuditEvent('input.rate_limited', { ...describeAction(action), rateLimitedTotal:
this.rateLimited }));
  }
}
}
}

```

Part D — Desktop: session audit hooks

FILE: apps/desktop/src/renderer/src/services/sessionAuditHooks.ts

```

import {
  isAuditDetailSafe,
  type SessionAuditEvent,
} from '@remotedesk/shared';

/**
 * Collects session audit events locally and optionally forwards them to a sink
 * (e.g. the Pack 14 AuditClient). Drops any event whose detail looks sensitive
 * (defense in depth) and never logs payloads. Forwarding failures are swallowed
 * so auditing never breaks a session.
 */
export type AuditSink = (event: SessionAuditEvent) => void | Promise<void>;

export class SessionAuditHooks {
  private buffer: SessionAuditEvent[] = [];

  constructor(private readonly sink?: AuditSink, private readonly maxBuffer = 500) {}

  /** Record an event. Sensitive-looking detail is rejected outright. */
  record(event: SessionAuditEvent): void {
    if (!isAuditDetailSafe(event.detail)) {
      console.warn('[audit] dropped event with unsafe detail', { type: event.type });
      return;
    }
    this.buffer.push(event);
    if (this.buffer.length > this.maxBuffer) this.buffer.shift();
    if (this.sink) {
      try {
        const r = this.sink(event);
        if (r instanceof Promise) r.catch(() => undefined);
      } catch {
        // Swallow: audit must never interrupt the session.
      }
    }
  }

  /** Recent events for display (newest last). */
  recent(limit = 50): SessionAuditEvent[] {
    return this.buffer.slice(-limit);
  }
}

```

```
}  
}
```

FILE: apps/desktop/src/renderer/src/services/emergencyStop.ts [MODIFY]

```
// ...existing imports from Pack 15...  
import { makeAuditEvent, type SessionAuditEvent } from '@remotedesk/shared';  
  
// ---- MODIFY: accept an optional audit callback in the constructor ----  
// constructor(  
//   private readonly permissionState: HostPermissionState,  
//   private readonly send: (frame: PermissionSnapshotFrame) => void,  
//   private readonly onAudit?: (e: SessionAuditEvent) => void, // ADD  
// ) {}  
  
// ---- MODIFY: emit audit on engage()/release() ----  
// engage(): void {  
//   ...existing...  
//   this.onAudit?.(makeAuditEvent('emergency_stop.engaged', { source: 'host' }));  
// }  
// release(): void {  
//   ...existing...  
//   this.onAudit?.(makeAuditEvent('emergency_stop.released', { source: 'host' }));  
// }
```

Part E — Desktop: route inbound input through the executor

FILE: apps/desktop/src/renderer/src/services/webrtc.ts [MODIFY]

```
// ...existing imports + Pack 15 remoteInputGate...  
import { HostInputExecutor } from './hostInputExecutor';  
import type { InputAction } from '@remotedesk/shared';  
  
// ---- ADD: a single host executor for the session ----  
// Construct with the existing remoteInputGate. nativeSink is OMITTED on purpose,  
// so execution is a safe no-op unless the project injects a real safe executor.  
export const hostInputExecutor = new HostInputExecutor(remoteInputGate, {  
  // nativeSink: existingSafeExecutor, // <-- ONLY if one already exists  
  // onAudit: (e) => sessionAuditHooks.record(e), // wire in RemoteSessionView  
});  
  
// ---- MODIFY: host inbound input handler now goes through the executor ----  
// Replace any direct gate check + execute with a single call:  
//  
// inputChannel.on('input', (frame) => {  
//   const action: InputAction = normalizeInputFrame(frame); // your existing normalize  
//   hostInputExecutor.execute(action); // gate + rate-limit + (no-op) native sink  
// });  
//  
// NOTE: the viewer-side outbound gate (guardOutbound) from Pack 15 stays exactly  
// as-is; this change is host-inbound only.
```

Part F — UI

FILE: apps/desktop/src/renderer/src/components/InputActivityMeter.tsx

```
import React, { useEffect, useState } from 'react';
import type { HostInputExecutor } from '../services/hostInputExecutor';

/**
 * Host-only diagnostics: shows executed / blocked / rate-limited input counts.
 * Read-only, non-sensitive (counts only — no key codes or coordinates).
 */
export function InputActivityMeter({ executor }: { executor: HostInputExecutor }) {
  const [stats, setStats] = useState(executor.stats());

  useEffect(() => {
    const id = setInterval(() => setStats(executor.stats()), 1000);
    return () => clearInterval(id);
  }, [executor]);

  return (
    <div className="input-activity-meter" role="status">
      <span className="iam_ok">executed {stats.executed}</span>
      <span className="iam_sep"> · </span>
      <span className="iam_blocked">blocked {stats.blocked}</span>
      <span className="iam_sep"> · </span>
      <span className="iam_limited">rate-limited {stats.rateLimited}</span>
    </div>
  );
}
```

FILE: apps/desktop/src/renderer/src/components/RemoteSessionView.tsx [MODIFY]

```
// ...existing imports + Pack 15/16 additions...
import { InputActivityMeter } from '../InputActivityMeter';
import { hostInputExecutor } from '../services/webrtc';
import { SessionAuditHooks } from '../services/sessionAuditHooks';

// ---- ADD: construct audit hooks once at session start (host) ----
// Optionally forward to the Pack 14 AuditClient: new SessionAuditHooks((e) => auditClient.record(...))
// const sessionAuditHooks = useMemo(() => new SessionAuditHooks(), []);
//
// Wire the executor + emergency stop to the audit hooks:
//   hostInputExecutor was constructed in webrtc.ts; pass onAudit there OR set up
//   emergencyStop with: new EmergencyStop(permissionState, send, (e) => sessionAuditHooks.record(e))

// ---- ADD to render (host only): the activity meter near the emergency stop ----
// {isHost && <InputActivityMeter executor={hostInputExecutor} />}
//
// Existing PolicyStatusBanner, EmergencyStopButton, SessionApprovalPrompt,
// SessionExpiryBadge, and read-only viewer video all stay as-is.
```

Part G — Tests

FILE: tests/permissions/inputRateLimiter.test.ts


```

import { describe, it, expect } from 'vitest';
import { InputRateLimiter } from '../../apps/desktop/src/renderer/src/services/inputRateLimiter';

describe('InputRateLimiter', () => {
  it('allows a burst up to capacity then blocks', () => {
    // discrete rate 10/s, burst 2 => capacity 20
    const rl = new InputRateLimiter({ discreteRatePerSec: 10, burstFactor: 2 }, 0);
    let allowed = 0;
    for (let i = 0; i < 25; i++) if (rl.allow('key', 0)) allowed += 1;
    expect(allowed).toBe(20); // capacity
  });

  it('refills over time', () => {
    const rl = new InputRateLimiter({ discreteRatePerSec: 10, burstFactor: 1 }, 0);
    for (let i = 0; i < 10; i++) rl.allow('key', 0); // drain
    expect(rl.allow('key', 0)).toBe(false);
    // 1 second later => 10 tokens refilled
    expect(rl.allow('key', 1000)).toBe(true);
  });

  it('gives high-frequency kinds a larger bucket', () => {
    const rl = new InputRateLimiter({ discreteRatePerSec: 5, highFreqRatePerSec: 100, burstFactor: 1 }, 0);
    let moves = 0;
    for (let i = 0; i < 100; i++) if (rl.allow('mouse_move', 0)) moves += 1;
    expect(moves).toBe(100);
    // discrete bucket is independent + smaller
    let keys = 0;
    for (let i = 0; i < 100; i++) if (rl.allow('key', 0)) keys += 1;
    expect(keys).toBe(5);
  });
});

```

FILE: tests/permissions/hostInputExecutor.test.ts

```

import { describe, it, expect, vi } from 'vitest';
import { HostInputExecutor } from '../../apps/desktop/src/renderer/src/services/hostInputExecutor';
import { InputRateLimiter } from '../../apps/desktop/src/renderer/src/services/inputRateLimiter';
import type { InputAction } from '@remotedesk/shared';

function action(seq: number): InputAction {
  return { kind: 'key', payload: { code: 'redacted-in-test' }, seq };
}

/** Minimal gate stub matching RemoteInputGate.guardInbound(). */
function gateStub(allow: boolean) {
  return { guardInbound: () => allow } as any;
}

describe('HostInputExecutor', () => {
  it('blocks when the policy gate denies (fail closed)', () => {
    const exec = new HostInputExecutor(gateStub(false));
    expect(exec.execute(action(1))).toBe('blocked');
    expect(exec.stats().blocked).toBe(1);
    expect(exec.stats().executed).toBe(0);
  });
});

```

```

it('executes (no-op native) when gate allows and under rate limit', () => {
  const exec = new HostInputExecutor(gateStub(true));
  expect(exec.execute(action(1))).toBe('executed');
  expect(exec.stats().executed).toBe(1);
});

it('rate-limits a flood', () => {
  const exec = new HostInputExecutor(gateStub(true), {
    rateLimiter: new InputRateLimiter({ discreteRatePerSec: 5, burstFactor: 1 }, 0),
  });
  const dispositions = Array.from({ length: 10 }, (_, i) => exec.execute(action(i), 0));
  expect(dispositions.filter((d) => d === 'executed')).toHaveLength(5);
  expect(dispositions.filter((d) => d === 'rate_limited')).toHaveLength(5);
});

it('forwards to a native sink ONLY when injected', () => {
  const sink = { execute: vi.fn() };
  const exec = new HostInputExecutor(gateStub(true), { nativeSink: sink });
  exec.execute(action(1));
  expect(sink.execute).toHaveBeenCalledTimes(1);

  const noSink = new HostInputExecutor(gateStub(true)); // default: no-op
  expect(noSink.execute(action(1))).toBe('executed'); // executes without throwing
});

it('emits a non-sensitive audit event on block', () => {
  const onAudit = vi.fn();
  const exec = new HostInputExecutor(gateStub(false), { onAudit });
  exec.execute(action(42));
  expect(onAudit).toHaveBeenCalledTimes(1);
  const event = onAudit.mock.calls[0][0];
  expect(event.type).toBe('input.blocked');
  expect(event.detail.seq).toBe(42);
  expect(JSON.stringify(event)).not.toContain('payload');
});
});

```

FILE: tests/permissions/sessionAuditEvents.test.ts

```

import { describe, it, expect } from 'vitest';
import { makeAuditEvent, isAuditDetailSafe } from '@remotedesk/shared';

describe('sessionAuditEvents', () => {
  it('builds an event with an ISO timestamp', () => {
    const e = makeAuditEvent('control.granted', { by: 'host' }, new Date('2026-06-15T10:00:00Z'));
    expect(e.type).toBe('control.granted');
    expect(e.at).toBe('2026-06-15T10:00:00.000Z');
    expect(e.detail.by).toBe('host');
  });

  it('flags unsafe detail keys', () => {
    expect(isAuditDetailSafe({ count: 3, reason: 'policy' })).toBe(true);
    expect(isAuditDetailSafe({ token: 'x' })).toBe(false);
    expect(isAuditDetailSafe({ payload: 'x' })).toBe(false);
    expect(isAuditDetailSafe({ authorization: 'x' })).toBe(false);
  });
});

```

```
});  
});
```

Part H — Reports

FILE: [generated-pack17-desktop-input-executor-merge-summary.md](#)

```
# Pack 17 (Desktop) — Safe Host Input Executor + Rate Limiting + Audit Hooks · Merge Summary
```

```
## What this slice does
```

Continues Desktop Packs 15 + 16. Adds the host-side choke point: every inbound input action funnels through ONE executor that (1) re-checks the Pack 15 policy gate, (2) rate-limits floods with a token bucket, and (3) only then hits a native sink — which is a NO-OP unless a safe executor is explicitly injected. Plus structured, non-sensitive session audit events (control grant/revoke, emergency stop, blocked/rate-limited input, expiry).

```
## Files created (9)
```

- packages/shared/src/permissions/inputAction.ts
- packages/shared/src/permissions/sessionAuditEvents.ts
- apps/desktop/src/renderer/src/services/hostInputExecutor.ts
- apps/desktop/src/renderer/src/services/inputRateLimiter.ts
- apps/desktop/src/renderer/src/services/sessionAuditHooks.ts
- apps/desktop/src/renderer/src/components/InputActivityMeter.tsx
- tests/permissions/inputRateLimiter.test.ts
- tests/permissions/hostInputExecutor.test.ts
- tests/permissions/sessionAuditEvents.test.ts

```
## Files modified (4)
```

- packages/shared/src/permissions/index.ts — export inputAction + sessionAuditEvents.
- apps/desktop/src/renderer/src/services/webRTC.ts — route host inbound input through HostInputExecutor.
- apps/desktop/src/renderer/src/services/emergencyStop.ts — emit audit on engage/release.
- apps/desktop/src/renderer/src/components/RemoteSessionView.tsx — input activity meter + audit hook wiring.

```
## Files intentionally NOT touched
```

- apps/desktop/src/main/index.ts, preload/index.ts — no real native executor enabled, no new bridge.
- socket.ts, permissionState.ts — unchanged from Pack 15/16.
- apps/api/**, apps/web/** — audit is session-local (optionally forwarded to Pack 14 client).

```
## Remaining manual wiring risks
```

1. Replace the host inbound input handler with a single hostInputExecutor.execute(action) call.
2. Provide your existing normalizeInputFrame(frame) -> InputAction mapping.
3. Inject a nativeSink ONLY if a vetted safe executor already exists; default stays no-op.
4. Construct SessionAuditHooks at session start; pass onAudit into the executor + emergency stop.
5. Optionally forward audit events to the Pack 14 AuditClient.

```
## Verify
```

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: flood input => meter shows rate-limited climbing, executed capped; policy off => blocked climbs; native still no-op.

FILE: [generated-pack17-desktop-input-executor-code-review.md](#)

```
# Pack 17 (Desktop) — Code Review Notes
```

Strengths

- ONE host choke point for inbound input: gate -> rate limit -> (no-op) native sink, in that fail-closed order.
- Native execution is off by default; a sink must be explicitly injected, so this pack cannot enable OS input on its own.
- Rate limiter is a pure token bucket, deterministic under an injected clock, separate buckets for high-freq vs discrete.
- Audit events are non-sensitive by construction (kind + seq + counts); a key-pattern guard rejects unsafe detail.
- Executor + rate limiter + audit all unit-tested without a live peer connection.

Reviewer checklist

- [] Host inbound handler routes through `hostInputExecutor.execute` (no direct execution left).
- [] `normalizeInputFrame` maps your wire frames to `InputAction` (kind + opaque payload + seq).
- [] `nativeSink` left undefined unless a vetted safe executor exists.
- [] Audit detail never includes payloads/keys (guard + `describeAction` enforce this).
- [] Rate limits tuned for your input cadence (defaults: 25/s discrete, 200/s high-freq, burst x2).

Safety

- Fail closed: gate denial blocks before rate limit; no permission set => Pack 15 gate already denies.
- No payloads, key codes, or coordinates logged or audited.
- Native OS execution remains disabled/no-op by default.

FILE: generated-pack17-desktop-input-executor-manifest.json

```
{
  "pack": "pack17-desktop-input-executor",
  "title": "Safe Host Input Executor Shell + Per-Action Rate Limiting + Session Audit Hooks",
  "actualFileCount": 13,
  "filesCreated": [
    "packages/shared/src/permissions/inputAction.ts",
    "packages/shared/src/permissions/sessionAuditEvents.ts",
    "apps/desktop/src/renderer/src/services/hostInputExecutor.ts",
    "apps/desktop/src/renderer/src/services/inputRateLimiter.ts",
    "apps/desktop/src/renderer/src/services/sessionAuditHooks.ts",
    "apps/desktop/src/renderer/src/components/InputActivityMeter.tsx",
    "tests/permissions/inputRateLimiter.test.ts",
    "tests/permissions/hostInputExecutor.test.ts",
    "tests/permissions/sessionAuditEvents.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/desktop/src/renderer/src/services/webrtc.ts",
    "apps/desktop/src/renderer/src/services/emergencyStop.ts",
    "apps/desktop/src/renderer/src/components/RemoteSessionView.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/src/main/index.ts",
    "apps/desktop/src/preload/index.ts",
    "apps/desktop/src/renderer/src/services/socket.ts",
    "apps/desktop/src/renderer/src/services/permissionState.ts",
    "apps/api/**",
    "apps/web/**"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "react", "vitest"],
    "assumedFromProject": [
      "Desktop Pack 15 RemoteInputGate (guardInbound)",
      "Desktop Pack 15 EmergencyStop",
    ]
  }
}
```

```

    "existing host inbound input handler + a frame->InputAction normalizer",
    "optional: Pack 14 AuditClient for forwarding"
  ]
},
"knownLimitations": [
  "Native OS input execution remains disabled/no-op (sink must be explicitly injected).",
  "Audit events are session-local unless forwarded to the Pack 14 AuditClient.",
  "Rate-limit defaults are heuristic; tune to real input cadence.",
  "normalizeInputFrame mapping is project-specific and must be supplied on merge."
],
"remainingManualWiringRisks": [
  "Route host inbound input through hostInputExecutor.execute exclusively.",
  "Supply normalizeInputFrame(frame) -> InputAction.",
  "Wire SessionAuditHooks into the executor + emergency stop (and optionally Pack 14).",
],
"estimatedCompletionAfterCarefulMerge": "64-70%"
}

```

Desktop Pack 17 done: 13 files (9 created, 4 modified). The host now has a single, auditable choke point for inbound input: every action runs gate -> rate limit -> (no-op) native sink, in that fail-closed order. A token-bucket limiter blunts input floods (separate buckets for mouse-move/scroll vs keys), and structured, non-sensitive audit events cover control grant/revoke, emergency stop, blocked, rate-limited, and expiry.

The clean bit: native OS execution stays off by default, a real sink must be *explicitly injected*, so this pack physically cannot enable OS input on its own. Audit + logs only ever carry action kind + sequence + counts, never payloads, key codes, or coordinates, enforced by `describeAction` plus a key-pattern guard that rejects unsafe audit detail. Everything funnels through Pack 15's existing gate first, so the choke point can never be looser than the policy.

Next slice = host-to-API audit forwarding + session-event timeline UI, or viewer-side reconnect-with-policy-recheck. Bolo.